

CODEBASE DOCUMENTATION

FUELSYNC

HYBRID ATHLETE OS



A complete, file-by-file guide to the FuelSync web application — what every file does, why it exists, and how each piece connects to the rest of the system.

■ [FOODANIEL.DANIELMMS.SITE](https://foodaniel.danielmms.site) ■ [REACT + TYPESCRIPT + VITE](#) ■ MAY 2026

Table of Contents

EVERY FILE, EXPLAINED

CHAPTER 1 — THE BIG PICTURE

What is FuelSync? App overview, tech stack, how it's deployed

File Map Every file and where it lives

CHAPTER 2 — INFRASTRUCTURE FILES

Dockerfile How the app is packaged into a container

nginx.conf Web server configuration — routing and proxy

CHAPTER 3 — ENTRY POINTS

main.tsx Where React starts

index.css Global styles and the design system

App.tsx Root shell — navigation, auth gate, Strava OAuth

CHAPTER 4 — SCREENS

HomeScreen.tsx Dashboard — calories, macros, training, weekly load

FoodScreen.tsx Food log — AI smart mode + manual entry

ProfileSetupScreen.tsx User stats, activity level, sign out

AuthScreen.tsx Login, register, forgot & reset password

CHAPTER 5 — COMPONENTS

TrainingPicker.tsx 4-button training type selector

StravaCard.tsx Strava integration card with run list

WeatherBanner.tsx Conditions display with training alert

MacroBar.tsx Horizontal macro progress bar (unused)

MacroCard.tsx Macro stat card with bar (unused)

RingProgress.tsx Circular SVG progress ring (unused)

CHAPTER 6 — API LAYER

api/client.ts Core HTTP client and token management

api/auth.ts Login, register, profile endpoints

api/food.ts Food log and AI estimation endpoints

api/strava.ts Strava OAuth and stats endpoints

CHAPTER 7 — STATE MANAGEMENT

store/authStore.ts Who is logged in

store/nutritionStore.ts Today's training, targets, weekly load

hooks/useNutrition.ts Bridge between store, profile, and engine

CHAPTER 8 — UTILITIES

utils/sounds.ts Web Audio API success sound

utils/runShareCard.ts Canvas-drawn run share card for Instagram

CHAPTER 9 — BUSINESS LOGIC ENGINE

nutritionEngine.ts TDEE, macro cycling, fatigue gate, adherence score

weatherService.ts Weather fetch and training condition alerts

CHAPTER 10 — SHARED TYPES & TESTS

shared/types/index.ts TypeScript interfaces used across the whole app

nutritionEngine.test.ts Manual smoke-tests for the nutrition engine

The Big Picture

● What is FuelSync?

FuelSync is a **Progressive Web App (PWA)** for hybrid athletes — people who combine running, strength training, and cardio. It lives at `foodaniel.danielmms.site` and runs on Daniel's home server inside a Docker container.

The app has three core jobs:

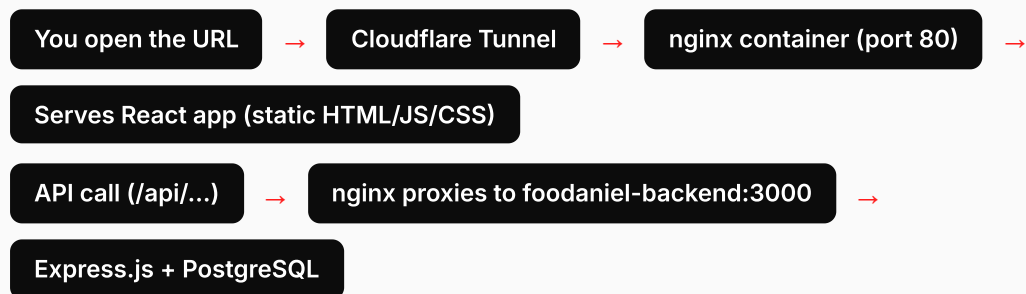
- **Tell you what to eat today** based on what type of training you're doing (rest, strength, cardio, or hybrid). The macro targets change every day.
- **Let you log food** either by typing a description in plain English (AI fills the macros for you) or by entering the numbers manually.
- **Show you your training load** — how many kilometers you've run, fatigue score, recovery, and your Strava data.

TECH STACK

LAYER	TECHNOLOGY	WHY
Frontend framework	React 18 + TypeScript	Component-based UI with type safety
Build tool	Vite	Fast dev server, optimized production build
State management	Zustand	Lightweight, no boilerplate, persists to localStorage
HTTP client	Native fetch (custom wrapper)	No library needed for simple REST calls
Styling	Inline styles + global CSS	Full control, no CSS-in-JS overhead
Web server	nginx	Serves static files, proxies /api to backend
Container	Docker	Runs alongside 15+ other services on the home server

AI	Google Gemini API	Estimates food macros from free-text descriptions
Running data	Strava API	OAuth to pull run stats and recent activities
Weather	OpenWeatherMap API	Gives training condition alerts
Backend	Express.js (separate container)	Handles auth, food logs, Strava OAuth, AI calls
Database	PostgreSQL 16	Stores users, food_logs, training_logs

HOW THE APP IS DEPLOYED



THE COMPLETE FILE MAP

```

FuelSync/
├── Dockerfile           ← How to build the Docker image
├── nginx.conf           ← Web server config
├── apps/web/src/
│   ├── main.tsx         ← React entry point
│   ├── App.tsx          ← Root component (nav + auth)
│   └── index.css        ← Global styles / design system
├── screens/
│   ├── AuthScreen.tsx   ← Login / Register / Reset
│   ├── HomeScreen.tsx   ← Dashboard
│   ├── FoodScreen.tsx   ← Food logging
│   └── ProfileSetupScreen.tsx ← User profile + settings
├── components/
│   ├── TrainingPicker.tsx ← Training type buttons
│   ├── StravaCard.tsx     ← Strava integration
│   ├── WeatherBanner.tsx  ← Weather + training alert
│   ├── MacroBar.tsx       ← (built, not used in UI yet)
│   ├── MacroCard.tsx      ← (built, not used in UI yet)
│   └── RingProgress.tsx   ← (built, not used in UI yet)

```

```

├── api/
│   ├── client.ts      ← HTTP fetch wrapper + token
│   ├── auth.ts        ← Auth endpoints
│   ├── food.ts        ← Food log endpoints
│   └── strava.ts       ← Strava endpoints
├── store/
│   ├── authStore.ts   ← Who is logged in (Zustand)
│   └── nutritionStore.ts ← Training day + weekly load (Zustand)
├── hooks/
│   └── useNutrition.ts ← Connects store ↔ engine ↔ weather
├── utils/
│   ├── sounds.ts      ← Food log success sound
│   └── runShareCard.ts ← Canvas image generator for sharing
├── apps/mobile/src/services/
│   ├── nutritionEngine.ts ← TDEE + macro cycling engine
│   └── weatherService.ts  ← Weather fetch + alert evaluator
├── shared/types/
│   ├── index.ts        ← All TypeScript interfaces
│   └── nutritionEngine.test.ts ← Manual tests for the engine

```

Infrastructure Files

● Dockerfile

Tells Docker how to build and package the FuelSync frontend into a deployable container image.

This file uses a **multi-stage build** — a common Docker pattern to keep the final image small by separating the "build step" from the "serve step".

STAGE 1 — BUILDER

- Starts from `node:20-alpine` (a tiny Linux image with Node.js)
- Copies `apps/web` (the React source), `apps/mobile/src/services` (the business logic), and `shared` (types)
- Runs `npm run build` — Vite compiles all TypeScript/JSX into plain HTML + JS + CSS in a `dist/` folder

STAGE 2 — SERVE

- Starts fresh from `nginx:alpine` — no Node.js at all
- Copies only the `dist/` folder from Stage 1
- Copies `nginx.conf` so nginx knows how to serve the app

WHY TWO STAGES?

The Node.js build tools (Vite, TypeScript compiler, all npm packages) weigh hundreds of megabytes. The final served app is just static HTML/CSS/JS — maybe a few hundred kilobytes. Two stages means the running container stays small and fast.

● nginx.conf

Configures the nginx web server — how to serve the React app and how to forward API requests to the backend.

TWO KEY RULES

- **Rule 1 — API proxy:** Any request starting with `/api/` is forwarded to `http://foodaniel-backend:3000/`. This is how the frontend talks to the Express backend without needing to know its real address.
- **Rule 2 — SPA fallback:** Every other URL serves `index.html`. This is essential for a Single Page App — if you refresh the page on `/profile`, nginx needs to return the React app (not a 404), and React Router handles the rest.

HOW API CALLS WORK END-TO-END

The React app calls `fetch('/api/food/logs')` → nginx receives it → proxies to `foodaniel-backend:3000/food/logs` → Express handles it → returns JSON → React updates the UI. The frontend never knows the backend's real port.

gzip compression is also enabled, which makes the app load faster by compressing text-based files (JS, CSS, HTML) before sending them to the browser.

Entry Points

● main.tsx

The very first file React executes. It mounts the entire app onto the HTML page.

It does exactly one thing: finds the `<div id="root">` in `index.html` and renders the `<App />` component inside it. `StrictMode` is a React development tool that helps catch potential bugs by running some code twice in development (has no effect in production).

Think of this as the "power button" of the app.

● index.css

The global design system — the visual DNA of the entire app. Every screen inherits from this file.

CSS VARIABLES (DESIGN TOKENS)

At the top, a set of named colors and animation curves are defined so they can be reused everywhere:

VARIABLE	VALUE	USED FOR
<code>--red</code>	<code>#FF1C1C</code>	Accent, CTAs, active states, bars
<code>--black</code>	<code>#0C0C0C</code>	Page background
<code>--card</code>	<code>#141414</code>	Card backgrounds
<code>--chrome</code>	<code>#C0C0C0</code>	Label text, subdued content
<code>--spring</code>	<code>cubic-bezier(0.34,1.56,0.64,1)</code>	Bouncy button press animation
<code>--ease</code>	<code>cubic-bezier(0.4,0,0.2,1)</code>	Smooth progress bar transitions

TYPOGRAPHY CLASSES

- `.nrc-hero` — Inter Black (weight 900), letter-spacing -2px, tight line-height. Used for big numbers like calorie counts.
- `.nrc-label` — Inter Bold, 10px, uppercase, 3px letter-spacing, silver color. Used for section headers like "CALORIES TODAY".

ANIMATION CLASSES

When any screen opens, its sections animate upward from below. The class `.nrc-a` defines the animation, and `.nrc-a1` through `.nrc-a6` add increasing delays (0, 60, 120, 180, 240, 300ms). This creates a staggered entrance effect where elements appear one after another.

INTERACTION CLASSES

- `.nrc-press` — Any element with this class scales down to 96% when tapped/clicked, giving physical button-like feedback. Also disables the blue tap highlight on mobile.

PROGRESS BAR CLASSES

- `.volt-bar-track` — The grey container bar (3px tall)
- `.volt-bar-fill` — The red fill that animates as macros are consumed

● App.tsx

The root shell of the entire app. Handles three things: deciding whether to show the login screen, routing between tabs, and processing the Strava OAuth callback.

AUTH GATE

On startup, it checks if a JWT token exists in localStorage. If yes, it calls `GET /auth/me` to fetch the user's profile. If that fails (expired token), it logs out and shows the login screen. If no token at all, it shows the login screen immediately.



TAB NAVIGATION

Three tabs: HOME, FUEL, STATS. The active tab shows a thin red line above it. Only one screen renders at a time — the others are simply not mounted. The nav bar is always 64px tall and sticks to the bottom.

STRAVA OAUTH CALLBACK

When Strava redirects back after authorization, it adds `?code= ... &scope= ...` to the URL. App.tsx detects this, calls `connectStrava(code)` to exchange the code for tokens, then removes the query params so the URL stays clean. A "Syncing Activities" splash screen shows during this step.

Screens

● HomeScreen.tsx

The main dashboard. Shows everything at a glance: calories consumed vs. target, macro progress, today's training type, weekly load stats, Strava runs, and weather conditions.

WHAT'S RENDERED (TOP TO BOTTOM)

SECTION	WHAT IT SHOWS	NOTES
Header	"Good Morning, DANIEL" + date	Greeting changes by time of day
Profile Alert	Red banner to complete profile	Only shows if weight/height/age are missing
Hero Calories	Big number + red progress bar	Turns red when you hit 100%
Macro Grid	Protein / Carbs / Fat / Recovery (2×2)	Each has its own bar and target
Training Mode	TrainingPicker + RESET button	Selection triggers macro recalculation
Workout Time	Time input	Only shows for non-rest days
Weekly Stats	KM Run / Sets / Fatigue + LOG RUN	Fatigue ≥ 70% shows a red warning
Running	StravaCard component	Shows connected/disconnected state
Conditions	WeatherBanner component	Only renders if weather data exists

HOW THE CALORIE DATA GETS HERE

On mount, the screen calls `getLogs(today)` to fetch all food entries for today from the backend. It then adds up all the calories, protein, carbs, and fat to get the `consumed` totals. This is compared against `targets` (calculated by the nutrition engine based on your training type) to show percentage progress.

KEY DETAIL — LEG FATIGUE GATE

When you tap a training type, the app checks your `legFatigueScore`. If it's ≥ 70 and you select "Cardio", a browser `confirm()` dialog appears asking if you want to override the recommendation. You can still proceed — it's a warning, not a hard block.

● FoodScreen.tsx

The food logging screen — where you add everything you eat. The most complex screen in the app.

THE MAIN VIEW

Shows calories consumed today (big hero number + bar), a 3-column macro row (protein/carbs/fat), then the food log grouped by meal type (Breakfast, Pre-Workout, Lunch, Post-Workout, Dinner, Snack). If nothing's been logged yet, a large faded "EMPTY" text appears with instructions.

Each food entry shows the name, macro breakdown (P/C/F in grams), calorie count, and a $\times$ button to delete it.

THE FAB BUTTON (+)

The red floating button in the bottom-right opens the **Add Fuel sheet** — a bottom drawer that slides up over the screen.

AI SMART MODE

The default mode when you tap +. You type a description in plain English — anything from "100g chicken breast" to "bowl of overnight oats with banana and honey". The app sends this to `POST /food/describe` on the backend, which calls the Gemini AI. Gemini returns estimated macros, a food name, weight, and a confidence level (high/medium/low). The form then switches automatically to Manual mode with all fields pre-filled so you can review and edit before saving.

MANUAL MODE

Enter the food name, weight in grams (or a text quantity like "2 eggs"), and macros manually. As you type protein/carbs/fat, the calorie field auto-calculates using $P \times 4 + C \times 4 + F \times 9$. There's also an "◆ AI" button that calls `POST /food/estimate` — you give it a name + weight, and it fills the macros for you.

VALIDATION

Before saving, four rules are checked:

- The calories field must match $P \times 4 + C \times 4 + F \times 9$ within $\pm 12\%$ (to catch typos)
- Total macros (P+C+F) cannot exceed the food weight $\times 1.1$ (physically impossible otherwise)
- No single macro can be more than 95% of the food weight (e.g., 95g protein from 100g of food)
- All fields must be filled and non-negative

ON SUCCESSFUL LOG

The entry is saved via `POST /food/logs`, the log list refreshes, the sheet closes, and `playFoodLogSound()` plays a short ascending chord.

● ProfileSetupScreen.tsx

Where users enter their body stats and set their activity level. This data powers the nutrition engine — without it, no macro targets can be calculated.

FIELDS

- **Display Name** — shown as the greeting on HomeScreen (e.g., "Good Morning, DANIEL"). Falls back to the email username if empty.
- **Weight (kg), Height (cm), Age** — used in the BMR formula
- **Biological Sex** — male adds +5 to BMR, female subtracts -161 (Mifflin-St Jeor)
- **Activity Level** — 5 options from Sedentary ($\times 1.2$) to Extra Active ($\times 1.9$), multiplied against BMR to get TDEE

STATS CARDS

If the profile is already filled in, three cards at the top show the current Weight/Height/Age values in red. This gives a quick summary without needing to scroll to the form.

SAVING

Calls `PATCH /auth/profile` and updates the Zustand auth store. The nutrition engine immediately recalculates macro targets using the new values. The Save button turns green with a "Saved ✓" state for 2.5 seconds after success.

ACCOUNT SECTION

Shows the logged-in email address and a "Sign Out" button. Sign out clears the JWT token from localStorage and resets the auth store — the app immediately shows the login screen.

● AuthScreen.tsx

The login/registration screen — the entry gate to the app. Handles four flows.

FLOW	ENDPOINT	WHAT HAPPENS
Login	POST /auth/login	Returns JWT token + user, stored in localStorage
Register	POST /auth/register	Creates account, returns JWT token + user
Forgot Password	POST /auth/forgot-password	Sends reset email via Resend API
Reset Password	POST /auth/reset-password	Validates reset token, sets new password

After a successful login or register, `setAuth(token, user)` is called on the auth store, which causes `App.tsx` to transition from AuthScreen to the main app.

Components

● TrainingPicker.tsx

A 2×2 grid of buttons for selecting today's training type. Pure presentational — receives the current selection and a callback, does nothing else.

OPTION	SUBTITLE	EFFECT ON MACROS
Rest	"High fat · Low carb"	Fat 50%, Carbs 20%, Protein 30%, Calories -15%
Strength	"High protein"	Protein 38%, Carbs 35%, Fat 27%, Calories +5%
Cardio	"High carb"	Carbs 55%, Protein 25%, Fat 20%, Calories +10%
Hybrid	"Balanced"	Protein 32%, Carbs 43%, Fat 25%, Calories +8%

The active button has a red left border (3px) and white text. Inactive buttons are dark grey. Tapping triggers the `onSelect` callback in HomeScreen, which runs the fatigue check and recalculates targets.

● StravaCard.tsx

Shows the Strava integration — either a connect button (if not linked) or your weekly stats and recent runs (if connected).

STATE: NOT CONNECTED

Shows a card with the Strava logo and a "Connect Strava" button. Tapping calls `GET /strava/auth-url` to get the OAuth redirect URL, then does `window.location.href = url` to send the user to Strava's authorization page.

STATE: CONNECTED

Shows three sections:

- **Weekly summary** — runs count, total km, total duration for the current week

- **YTD + All-Time** — year-to-date km and all-time km milestone
- **Recent Runs list** — each run shows name, date, distance, and chips for pace/time/elevation/heart rate

RUNROW — TWO ACTIONS PER RUN

- **LOG RUN** — adds the run's km to the weekly load in the Zustand store (updates fatigue score). Button turns green "✓ LOGGED" and disables.
- **SHARE** — calls `shareRunCard(run)` to generate and share/download the canvas card (see Chapter 8).

● WeatherBanner.tsx

Displays current weather conditions and a training recommendation alert. Only renders when weather data is available.

Shows temperature (large), weather description, city, humidity, and UV index. The alert level determines the left border color and a badge:

- **NONE** — green. "Conditions are good for outdoor training."
- **CAUTION** — yellow. Adjust pace, extra hydration shown in blue.
- **DANGER** — red. Extreme conditions, swap to indoor strength recommended.

If a training type swap is suggested, a dark "SWAP → STRENGTH" box appears. If extra hydration is recommended, the amount is shown in blue.

● MacroBar.tsx

⚠ Built but not used in UI

A horizontal progress bar for a single macro — label, current/target numbers, and a colored fill bar.

Takes `label`, `current`, `target`, `unit`, and `color` as props. Turns red with a glow effect when the current value exceeds the target. Animates with a 0.5s eased transition.

This component exists but is not currently used in any screen — it was likely built during an earlier design iteration.

● MacroCard.tsx

⚠ Built but not used in UI

A card version of the macro display with a large number, "of X unit" subtitle, a progress bar, and a "X left / +X over" footer.

Similar to MacroBar but in card form (with padding and rounded corners). Shows remaining vs. over in different colors. Not currently used — the HomeScreen uses an inline `MacroTile` component defined directly within that file instead.

● RingProgress.tsx

⚠ Built but not used in UI

A circular SVG ring that fills proportionally to show progress — like a fitness tracker ring.

Draws two SVG circles: a grey "track" ring underneath, and a colored "progress" arc on top. The arc is drawn using `strokeDasharray` — a CSS trick where you set the dash length to the percentage of the ring's circumference you want filled. The number is shown in the center. Turns red when progress exceeds 100%. Not currently used in the app.

API Layer

The API layer is a set of TypeScript files that handle all communication between the React frontend and the Express backend. Every network call in the app goes through these files.

● `api/client.ts`

The foundation of all API communication. Manages the auth token and provides a typed fetch wrapper.

TOKEN MANAGEMENT

The JWT token is stored in `localStorage` under the key `fs_token`. The `token` object provides `.get()`, `.set(t)`, and `.clear()` methods. Any module that needs to read or modify the token imports from here — there's one single source of truth.

THE REQUEST() FUNCTION

This is the core fetch wrapper. It:

- Prepends `/api` to every path (so you write `/auth/login`, not `/api/auth/login`)
- Automatically adds `Content-Type: application/json` and `Authorization: Bearer <token>` to every request
- Parses the JSON response
- Throws a descriptive Error if the HTTP status is not OK (e.g., "Invalid credentials" from the server)

THE API OBJECT

Four shorthand methods built on top of `request()`:

```
api.get('/food/logs?date=2026-05-15') // GET
api.post('/food/logs', { food_name: 'Rice' }) // POST with body
api.patch('/auth/profile', { weightKg: 80 }) // PATCH with body
api.delete('/food/logs/123') // DELETE
```

● `api/auth.ts`

All authentication-related API calls. Also defines the `BackendUser` interface — the shape of the user object the server returns.

BACKENDUSER INTERFACE

This is what the server sends back for a logged-in user: `id`, `email`, `dailyGoal`, `displayName`, `weightKg`, `heightCm`, `age`, `gender`, `activityLevel`. This interface is used everywhere a user object is needed.

FUNCTIONS

- `register(email, password)` — creates a new account, automatically stores the returned token
- `login(email, password)` — logs in, automatically stores the returned token
- `getMe()` — fetches the current user's profile (called on app start to restore the session)
- `updateProfile(data)` — updates any combination of body stats or display name
- `forgotPassword(email)` — triggers a password reset email
- `resetPassword(token, password)` — sets a new password using a reset token from the email link

● `api/food.ts`

All food logging and AI estimation calls. Defines two key interfaces and five functions.

FOODLOG INTERFACE

Represents a single logged food entry from the database: `id`, `food_name`, `calories`, `protein`, `carbs`, `fat`, `weight_grams`, `meal_type`, `image_url`, `logged_at`.

AIESTIMATE INTERFACE

Represents what the AI returns after analyzing a food description: all the macros plus `confidence` (high/medium/low) and an optional `imageUrl` if the backend found a food image.

FUNCTIONS

- `getLogs(date)` — fetches all food entries for a specific date (e.g., "2026-05-15")
- `addLog(entry)` — saves a new food entry to the database
- `deleteLog(id)` — removes a food entry by ID
- `estimateByWeight(name, grams)` — sends food name + weight to AI, gets macros back
- `estimateByDescription(text)` — sends free text to AI, gets full estimate back

● `api/strava.ts`

All Strava integration calls — OAuth connection, fetching stats, and disconnecting.

STRAVARUN INTERFACE

A single run: `id`, `name`, `date`, `distanceKm`, `duration`, `pace`, `elevationM`, `hrAvg` (nullable — not all runs have heart rate data).

STRAVASTATS INTERFACE

The full stats response: athlete name + pic, weekly summary (runs/km/duration), year-to-date, all-time, and the list of recent runs.

FUNCTIONS

- `getStravaAuthUrl()` — gets the OAuth redirect URL to send the user to Strava
- `connectStrava(code)` — exchanges the OAuth code for tokens (called after redirect back)
- `getStravaStats()` — fetches current Strava data (returns either connected stats or `{connected: false}`)
- `disconnectStrava()` — removes Strava tokens from the database

State Management

FuelSync uses **Zustand** for global state. Zustand is a minimal state management library — think of it as a shared variable that any component can read or update, and when it changes, the components using it automatically re-render.

● store/authStore.ts

Stores who is currently logged in. The simplest store — just the user and token state.

STATE

- `user: BackendUser | null` — the full user object (or null if logged out)
- `hasToken: boolean` — whether a JWT token exists in localStorage (checked on startup)

ACTIONS

- `setAuth(token, user)` — called after login/register: saves token to localStorage, sets user
- `setUser(user)` — called after profile update: refreshes the user object in memory
- `logout()` — clears token from localStorage, sets user to null

NOT PERSISTED TO LOCALSTORAGE

The `user` object itself is not saved to localStorage — only the token is. On page refresh, the app uses the token to call `GET /auth/me` and re-fetch the user. This ensures the user data is always fresh from the server.

● store/nutritionStore.ts

Stores today's training data, calculated macro targets, weekly load statistics, and weather data. The most important store in the app.

STATE

- `todayLog: DailyLog | null` — today's training selection (type, planned time, intensity)
- `targets: MacroTargets | null` — calculated calorie + macro goals for today
- `weeklyLoad: WeeklyLoad` — running km, strength sets, fatigue score, recovery score
- `weather: WeatherConditions | null` — latest weather fetch
- `environmentAlert: EnvironmentAlert | null` — training recommendation from weather

ACTIONS

- `logWorkoutComplete(km, sets)` — updates weeklyLoad after finishing a workout (calls `updateWeeklyLoad()` from the engine)
- `addRunKm(km)` — adds km to the running total (used from StravaCard when you log a Strava run)
- `resetDay()` — clears todayLog and targets (the RESET button)

PERSISTENCE

The store uses Zustand's `persist` middleware to save to localStorage under the key `fuelsync-nutrition-v2`. Only `weeklyLoad` and `todayLog` are saved — targets and weather are not, since they're cheap to recalculate.

● hooks/useNutrition.ts

A React hook that acts as the bridge between the Zustand store, the user's profile, the nutrition engine, and the weather service. This is what screens use instead of touching the store and engine directly.

WHAT IT CONNECTS

`authStore (user)` + `nutritionStore` + `nutritionEngine` → `macro targets`

TOUSERPROFILE(USER)

Converts a `BackendUser` (what the server returns) into a `UserProfile` (what the nutrition engine expects). Returns null if the profile is incomplete (missing weight/height/age/gender).

AUTO-RECALCULATION EFFECT

A `useEffect` watches for changes to weight, height, age, gender, activity level, or training type. Whenever any of these change, it calls `computeMacros()` and saves the new targets to the store. This means saving your profile immediately updates the macro bars on the home screen.

LOGDAY(TYPE, TIME)

The function called when you tap a training type. It: (1) checks the leg fatigue gate, (2) creates a DailyLog object, (3) saves it to the store, (4) computes and saves new macro targets. Returns whether it was blocked and a message if so.

REFRESHWEATHER()

Uses the browser's Geolocation API to get your coordinates, calls `fetchWeather()` from the weather service, runs `evaluateEnvironment()` to generate a training alert, and saves both to the store. Does nothing if location permission is denied.

Utilities

● `utils/sounds.ts`

Plays a short success sound when you log a food entry.

Uses the Web Audio API — no audio files needed. It synthesizes a three-note ascending arpeggio:

- **C5** at 523 Hz (plays immediately)
- **E5** at 659 Hz (plays 100ms later)
- **G5** at 784 Hz (plays 200ms later)

Each note is a sine wave oscillator that ramps up in 20ms, then fades out exponentially over 320ms. The overall volume is quiet (18% of max). If the browser doesn't support Web Audio, it silently does nothing.

WHY THIS MATTERS

The three notes form a C major chord arpeggio — a musically satisfying "success" sound without needing to ship any audio files. It's completely generated on the fly from math.

● `utils/runShareCard.ts`

Generates a 1080×1350px image of a run summary (Instagram portrait format) and shares or downloads it. Entirely drawn with the Canvas API — no images needed.

GENERATERUNCARD(RUN)

Draws the card layer by layer:

- Black background with subtle diagonal speed lines and track arc decorations (very low opacity)
- Red top stripe (16px) and left accent stripe (8px)
- "FUELSYNC — HYBRID ATHLETE OS" header

- The distance in massive text (240px font) with a red glow shadow effect
- "KM" label below in red
- Run name and date
- A dark stats grid with Pace, Time, Elevation, and HR — each in a different color (blue/orange/green/red)
- "► VIA STRAVA" in Strava orange
- Footer with "FUELSYNC" branding

SHARERUNCARD(RUN)

Generates the canvas → converts to a PNG blob → if the device supports the Web Share API with files (most Android/iOS), it opens the native share sheet. If not (desktop), it triggers a file download instead.

Business Logic Engine

These two files contain the core intelligence of FuelSync — no UI, no network calls, just pure functions that take data and return calculated results. They live in `apps/mobile/src/services/` but are shared with the web app via Vite's path aliases.

● `nutritionEngine.ts`

The heart of FuelSync's nutrition logic. Calculates how many calories and macros you should eat today, based on your body stats and training type.

STEP 1 — BMR (BASAL METABOLIC RATE)

Uses the **Mifflin-St Jeor formula** — the most accurate standard BMR equation:

```
BMR = (10 × weight_kg) + (6.25 × height_cm) - (5 × age)
      + 5   (if male)
      - 161 (if female)
```

BMR = the calories your body burns just existing (breathing, heartbeat, etc.) — before any activity.

STEP 2 — TDEE (TOTAL DAILY ENERGY EXPENDITURE)

Multiplies BMR by an activity multiplier:

ACTIVITY LEVEL	MULTIPLIER	DESCRIPTION
Sedentary	×1.2	Desk job, no exercise
Light	×1.375	1–3 training days/week
Moderate	×1.55	3–5 training days/week
Very Active	×1.725	6–7 training days/week
Extra Active	×1.9	Twice-daily training

STEP 3 — TRAINING DAY ADJUSTMENT

TDEE is adjusted based on training type:

TYPE	CAL ADJUSTMENT	PROTEIN	CARBS	FAT
Rest	×0.85 (−15%)	30%	20%	50%
Strength	×1.05 (+5%)	38%	35%	27%
Cardio	×1.10 (+10%)	25%	55%	20%
Hybrid	×1.08 (+8%)	32%	43%	25%

STEP 4 — RECOVERY PENALTY

If your `recoveryScore` is below 40 (heavily fatigued), calories are reduced by 5% to prioritize tissue repair over performance.

CARB TIMING WINDOW

For non-rest days with a planned workout time, the engine calculates a ±2-hour window around the workout. 50–60% of daily carbs should be consumed in this window for maximum performance. Example: workout at 18:00 → eat most carbs between 16:00 and 20:00.

LEG FATIGUE GATE

If `legFatigueScore ≥ 70` and you select Cardio, the engine returns `{blocked: true, message: "..."}` . This is a protective mechanism — high leg fatigue combined with a hard cardio session risks injury.

WEEKLY LOAD TRACKING

After each workout is logged, `updateWeeklyLoad()` updates the fatigue and recovery scores:

- Cardio: leg fatigue += km × 1.5 (capped at +20)
- Strength: leg fatigue += sets × 1.2
- Rest: leg fatigue -= 15 (natural decay)
- Recovery score: +10 for rest/low intensity, −5 for moderate, −15 for high intensity

MACRO ADHERENCE SCORE

A 0–100 score of how closely you hit your targets. Weighted toward protein (most important for hybrid athletes):

```
score = protein_hit × 0.40
      + carbs_hit  × 0.30
```

```
+ fat_hit    × 0.15
+ cal_hit    × 0.15
```

● weatherService.ts

Fetches weather data and evaluates whether conditions are safe for the planned training type.

FETCHWEATHER(LAT, LON, APIKEY)

Calls the OpenWeatherMap API with your GPS coordinates. Returns temperature (°C), humidity (%), UV index, weather description, and city name.

EVALUATEENVIRONMENT(WEATHER, TRAININGTYPE)

The decision engine. Uses three thresholds:

CONDITION	THRESHOLD	ALERT LEVEL
Temperature	≥ 36°C	Danger
Temperature	≥ 30°C	Caution
Humidity	≥ 90%	Danger
Humidity	≥ 80%	Caution
UV Index	≥ 8	Caution

Danger recommends swapping to indoor strength, adds +1000ml hydration.

Caution recommends adjusting pace, adds +400–600ml hydration, may suggest swapping cardio to strength.

SUGGESTWEEKLYPIVOT()

If a day is blocked by weather, this function looks at the next 3 days in the schedule and swaps the blocked day with the nearest rest or strength day. Keeps the week's total training load intact while avoiding dangerous conditions.

Shared Types & Tests

● `shared/types/index.ts`

The single source of truth for all TypeScript interfaces used across the entire app. If a data structure appears in more than one file, it's defined here.

KEY TYPES EXPLAINED

TYPE	WHAT IT REPRESENTS
<code>TrainingType</code>	'rest' 'strength' 'cardio' 'hybrid'
<code>UserProfile</code>	Body stats (weight, height, age, sex, activity level) used by the nutrition engine
<code>DailyLog</code>	Today's training choice — type, planned time, intensity, notes
<code>MacroTargets</code>	Calorie + protein + carbs + fat goals for the day, plus optional carb timing window
<code>MacroBreakdown</code>	Full output from the engine — targets + TDEE + rationale text
<code>WeeklyLoad</code>	Running total of km, sets, leg fatigue score, recovery score
<code>WeatherConditions</code>	Temperature, humidity, UV, description, city, timestamp
<code>EnvironmentAlert</code>	Alert level + message + optional pivot suggestion + extra hydration
<code>FoodEntry</code>	A food log entry with macros, weight, timestamp, and meal slot
<code>DaySnapshot</code>	A complete picture of a day — log, targets, consumed, weather, weekly load

WHY A SHARED TYPES FILE?

Without this file, the same interfaces would be copy-pasted across multiple files. Any change (e.g., adding a new field to `WeeklyLoad`) would need to be made in every file

that uses it. With this file, you change it once and TypeScript flags every place that needs updating.

● `shared/types/nutritionEngine.test.ts`

Manual smoke tests for the nutrition engine. Verifies that the core logic produces correct results before deploying.

Run with: `npx ts-node shared/types/nutritionEngine.test.ts`

Tests covered:

- **TDEE range check** — for an 80kg, 180cm, 28-year-old very active male, TDEE should be 2500–3500 kcal
- **Rest day** — fat must be greater than carbs; no carb timing window
- **Strength day** — protein must be greater than fat; carb timing window must exist
- **Cardio day** — carbs must be greater than both protein and fat
- **Carb timing window** — workout at 07:00 → pre-window at 05:00, post-window at 09:00
- **Remaining calculator** — subtracts consumed from targets correctly
- **Perfect adherence** — score should be 100 when consumed equals targets
- **Leg fatigue gate** — cardio blocked at score 75, allowed at score 50
- **Weekly load update** — km added correctly, fatigue score increases after cardio

NOTE

This is a plain TypeScript test file, not a framework like Jest or Vitest. It uses a simple hand-written `assert(condition, message)` function. It throws on failure and prints "PASS" lines on success. Fast to run, zero dependencies.

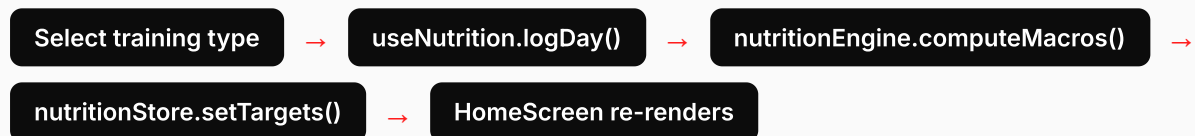
Quick Reference

● How Data Flows Through the App

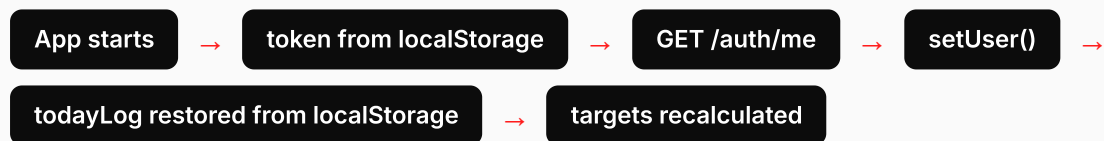
FOOD LOGGING FLOW



MACRO TARGET CALCULATION FLOW



SESSION RESTORE FLOW (ON PAGE RELOAD)



ALL API ENDPOINTS

METHOD	PATH	WHAT IT DOES
POST	/auth/register	Create new account
POST	/auth/login	Log in, get JWT
GET	/auth/me	Get current user profile
PATCH	/auth/profile	Update body stats / display name
POST	/auth/forgot-password	Send reset email
POST	/auth/reset-password	Set new password with reset token
GET	/food/logs?date=	Get all food entries for a date
POST	/food/logs	Add a new food entry

DELETE	/food/logs/:id	Delete a food entry
POST	/food/estimate	AI: name + grams → macros
POST	/food/describe	AI: free text → full estimate
GET	/strava/auth-url	Get Strava OAuth redirect URL
POST	/strava/connect	Exchange OAuth code for tokens
GET	/strava/stats	Get Strava stats and recent runs
DELETE	/strava/disconnect	Remove Strava tokens

LOCALSTORAGE KEYS

KEY	WHAT'S STORED
<code>fs_token</code>	JWT authentication token (30-day expiry)
<code>fuelsync-nutrition-v2</code>	todayLog + weeklyLoad (persisted between sessions)